

Reward, Cost, Termination, and Step Information



Following the standard OpenAI Gym API, after each step of the environment `env.step(...)`, the environment will return a tuple containing five items: `(obs, reward, terminated, truncated, info)`. In this page, we discuss the design of reward function `reward`, cost function `info["cost"]`, termination criterion `terminated` in various settings, truncation information `truncated`, and the details of step information `info`.

Reward Function

For all environments, reward functions consist of generally a dense driving reward and a sparse terminal reward. The dense reward is the longitudinal movement along the reference line or lane toward destination. When the episode is terminated due to, i.e. arriving the destination or driving out of the road, a sparse reward will be added to the dense reward. In practice, the concrete implementations of reward function are slightly different across all environments.

MetaDriveEnv

The reward functions for the `MetaDriveEnv` and derived environments like `Multi-agent Environments` and `SafeMetaDriveEnv` are similar as they all using `PGMap`. The implementation is as follows.

```
from metadrive.envs.metadrive_env import MetaDriveEnv
from metadrive.envs.scenario_env import ScenarioEnv
from metadrive.utils.doc_utils import print_source
print_source(MetaDriveEnv.reward_function)
```

```

def reward_function(self, vehicle_id: str):
    """
    Override this func to get a new reward function
    :param vehicle_id: id of BaseVehicle
    :return: reward
    """
    vehicle = self.agents[vehicle_id]
    step_info = dict()

    # Reward for moving forward in current lane
    if vehicle.lane in vehicle.navigation.current_ref_lanes:
        current_lane = vehicle.lane
        positive_road = 1
    else:
        current_lane = vehicle.navigation.current_ref_lanes[0]
        current_road = vehicle.navigation.current_road
        positive_road = 1 if not current_road.is_negative_road() else -1
    long_last, _ = current_lane.local_coordinates(vehicle.last_position)
    long_now, lateral_now = current_lane.local_coordinates(vehicle.position)

    # reward for lane keeping, without it vehicle can learn to overtake but fail to keep in
    lane
    if self.config["use_lateral_reward"]:
        lateral_factor = clip(1 - 2 * abs(lateral_now) /
vehicle.navigation.get_current_lane_width(), 0.0, 1.0)
    else:
        lateral_factor = 1.0

    reward = 0.0
    reward += self.config["driving_reward"] * (long_now - long_last) * lateral_factor *
positive_road
    reward += self.config["speed_reward"] * (vehicle.speed_km_h / vehicle.max_speed_km_h) *
positive_road

    step_info["step_reward"] = reward

    if self._is_arrive_destination(vehicle):
        reward = +self.config["success_reward"]
    elif self._is_out_of_road(vehicle):
        reward = -self.config["out_of_road_penalty"]
    elif vehicle.crash_vehicle:
        reward = -self.config["crash_vehicle_penalty"]
    elif vehicle.crash_object:
        reward = -self.config["crash_object_penalty"]
    elif vehicle.crash_sidewalk:
        reward = -self.config["crash_sidewalk_penalty"]
    step_info["route_completion"] = vehicle.navigation.route_completion

    return reward, step_info

```

This reward function is composed of three parts as follows:

$$R = c_1 R_{driving} + c_2 R_{speed} + R_{termination}$$

- The **driving reward** $R_{driving} = d_t - d_{t-1}$, wherein the d_t and d_{t-1} denote the longitudinal coordinates of the target vehicle on the current reference lane of two consecutive steps, providing dense reward to encourage agent to move toward the destination.
- The **speed reward** $R_{speed} = v_t / v_{max}$ incentivizes agent to drive fast. v_t and v_{max} denote the current velocity and the maximum velocity (80 km/h), respectively.

- The **termination reward** $R_{termination}$ contains a set of sparse rewards. At the end of episode, other dense rewards will be disabled and only one sparse reward will be given to the agent at the end of the episode according to its termination state. We implement the `success_reward`, `out_of_road_penalty`, `crash_vehicle_penalty` and `crash_object_penalty` currently. The penalty will be given as negative reward.

We also provide a config call `use_lateral_reward`, which is a multiplier in range $[0, 1]$ indicating whether the ego vehicle is far from the center of current lane. The multiplier will apply to the driving reward.

We summarize the default reward config here:

- `success_reward = 10.0`: one of termination reward.
- `out_of_road_penalty = 5.0`: will use -5.0 as the termination reward.
- `crash_vehicle_penalty = 5.0`: will use -5.0 as the termination reward.
- `crash_object_penalty = 5.0`: will use -5.0 as the termination reward.
- `driving_reward = 1.0`: the c_1 in reward function.
- `speed_reward = 0.1`: the c_2 in reward function.
- `use_lateral_reward = False`: disable weighting the driving reward according to centering in the lane.

ScenarioEnv

The reward function for `ScenarioEnv` is similar to the one of `MetaDriveEnv`, while the calculation of driving reward is slightly different and there are more items. The concrete implementation is:

```
print_source(ScenarioEnv.reward_function)
```

```

def reward_function(self, vehicle_id: str):
    """
    Override this func to get a new reward function
    :param vehicle_id: id of BaseVehicle
    :return: reward
    """
    vehicle = self.agents[vehicle_id]
    step_info = dict()

    # Reward for moving forward in current lane
    current_lane = vehicle.lane
    long_last = vehicle.navigation.last_longitude
    long_now = vehicle.navigation.current_longitude
    lateral_now = vehicle.navigation.current_lateral

    # dense driving reward
    reward = 0
    reward += self.config["driving_reward"] * (long_now - long_last)

    # reward for lane keeping, without it vehicle can learn to overtake but fail to keep in
    lane
    lateral_factor = abs(lateral_now) / self.config["max_lateral_dist"]
    lateral_penalty = -lateral_factor * self.config["lateral_penalty"]
    reward += lateral_penalty

    # heading diff
    ref_line_heading = vehicle.navigation.current_heading_theta_at_long
    heading_diff = wrap_to_pi(abs(vehicle.heading_theta - ref_line_heading)) / np.pi
    heading_penalty = -heading_diff * self.config["heading_penalty"]
    reward += heading_penalty

    # steering_range
    steering = abs(vehicle.current_action[0])
    allowed_steering = (1 / max(vehicle.speed, 1e-2))
    overflowed_steering = min((allowed_steering - steering), 0)
    steering_range_penalty = overflowed_steering * self.config["steering_range_penalty"]
    reward += steering_range_penalty

    if self.config["no_negative_reward"]:
        reward = max(reward, 0)

    # crash penalty
    if vehicle.crash_vehicle:
        reward = -self.config["crash_vehicle_penalty"]
    if vehicle.crash_object:
        reward = -self.config["crash_object_penalty"]
    if vehicle.crash_human:
        reward = -self.config["crash_human_penalty"]
    # lane line penalty
    if vehicle.on_yellow_continuous_line or vehicle.crash_sidewalk or
vehicle.on_white_continuous_line:
        reward = -self.config["on_lane_line_penalty"]

    step_info["step_reward"] = reward

    # termination reward
    if self._is_arrive_destination(vehicle):
        reward = self.config["success_reward"]
    elif self._is_out_of_road(vehicle):
        reward = -self.config["out_of_road_penalty"]

    # TODO LQY: all a callback to process these keys
    step_info["track_length"] = vehicle.navigation.reference_trajectory.length

```

```

step_info["carsize"] = [vehicle.WIDTH, vehicle.LENGTH]
# add some new and informative keys
step_info["route_completion"] = vehicle.navigation.route_completion
step_info["curriculum_level"] = self.engine.current_level
step_info["scenario_index"] = self.engine.current_seed
step_info["num_stored_maps"] = self.engine.map_manager.num_stored_maps
step_info["scenario_difficulty"] = self.engine.data_manager.current_scenario_difficulty
step_info["data_coverage"] = self.engine.data_manager.data_coverage
step_info["curriculum_success"] = self.engine.curriculum_manager.current_success_rate
step_info["curriculum_route_completion"] =
self.engine.curriculum_manager.current_route_completion
step_info["lateral_dist"] = lateral_now

step_info["step_reward_lateral"] = lateral_penalty
step_info["step_reward_heading"] = heading_penalty
step_info["step_reward_action_smooth"] = steering_range_penalty
return reward, step_info

```

This reward function is composed of six parts as follows:

$$R = c_1 R_{driving} + c_2 R_{lateral} + c_3 R_{heading} + c_4 R_{steering} - c_5 P_{collision} + R_{termination}$$

- The **driving reward** $R_{driving} = d_t - d_{t-1}$, wherein the d_t and d_{t-1} denote the longitudinal movements of the target vehicle on the recorded trajectory of two consecutive time steps, providing dense reward to encourage agent to move toward the destination.
- The **lateral reward** $R_{lateral}$ incentives the agent to driving as close to the reference trajectory as possible. When the distance between the car and the reference trajectory is larger than `max_lateral_dist`, the episode will be terminated due to driving out of road.
- The **heading reward** $R_{heading}$ asks the agent to have the same heading as the direction of a certain point on the lane. The point is calculated per step by projecting the position of the vehicle to the reference line (recorded trajectory).
- The **steering reward** $R_{steering}$ is for yielding large steering actions when the speed is high. The higher the speed is, the larger the penalty for the large steering angle will be.
- The **collision penalty** $P_{collision}$ is a constant scalar for penalizing any collision behaviors with certain type of traffic participants.
- The **termination reward** $R_{termination}$ contains a set of sparse rewards. At the end of episode, other dense rewards will be disabled and only one sparse reward will be given to the agent at the end of the episode according to its termination state. We use the `success_reward`, `out_of_road_penalty` for this environment. The penalty will be given as negative reward.

There is a special config `no_negative_reward`. When setting it to `True`, the final reward will be clipped to `reward=max(reward, 0)`. This is helpful for stabilizing the training. We summarize the default reward config of the `ScenarioEnv` here:

- `success_reward=5.0`,
- `out_of_road_penalty=5.0`,
- `on_lane_line_penalty=1.`,
- `crash_vehicle_penalty=1.`,

- `crash_object_penalty=1.0`,
- `crash_human_penalty=1.0`,
- `driving_reward=1.0`,
- `steering_range_penalty=0.5`,
- `heading_penalty=1.0`,
- `lateral_penalty=.5`,
- `max_lateral_dist=4`,
- `no_negative_reward=True`,

It is worth noting that we write some information like the lateral reward and heading reward to the `step_info`. Thus we can keep track of the training status of the agent.

Cost Function

Similar to the reward function, we also provide default cost function to measure the safety during driving. The cost function will be placed in the returned information dict as

`info["cost"]` after `env.step` function.

- `crash_vehicle_cost = 1.0`: yield cost when crashing to other vehicles.
- `crash_human_cost = 1.0`: yield cost when crashing to other vehicles.
- `crash_object_cost = 1.0`: yield cost when crashing to objects, such as cones and triangles.
- `out_of_road_cost = 1.0`: yield cost when driving out of the road.

The implementation of cost function is simple and almost the same for `MetaDriveEnv` and `ScenarioEnv`:

```
from metadrive.utils.doc_utils import print_source
from metadrive.envs import MetaDriveEnv
print_source(MetaDriveEnv.cost_function)
```

```
def cost_function(self, vehicle_id: str):
    vehicle = self.agents[vehicle_id]
    step_info = dict()
    step_info["cost"] = 0
    if self._is_out_of_road(vehicle):
        step_info["cost"] = self.config["out_of_road_cost"]
    elif vehicle.crash_vehicle:
        step_info["cost"] = self.config["crash_vehicle_cost"]
    elif vehicle.crash_object:
        step_info["cost"] = self.config["crash_object_cost"]
    return step_info['cost'], step_info
```

You can modify this function to add more information to the `step_info` dict. For example, you can log what kind of object raises this cost. Thus you can calculate how many vehicle collides with in one episode by summing up the number of vehicle crashes in each step.

Termination and Truncation

MetaDrive will terminate an episode of a vehicle if:

1. the target vehicle arrive its destination,
2. the vehicle drives out of the road,
3. the vehicle crashes to other vehicles,
4. the vehicle crashes to obstacles,
5. the vehicle crashes to human,
6. reach max step (horizon) limits, or
7. the vehicle crashes to building (e.g. in Multi-agent Tollgate environment).

The above termination function is implemented as:

```
print_source(MetaDriveEnv.done_function)
```

```

def done_function(self, vehicle_id: str):
    vehicle = self.agents[vehicle_id]
    done = False
    max_step = self.config["horizon"] is not None and self.episode_lengths[vehicle_id] >=
self.config["horizon"]
    done_info = {
        TerminationState.CRASH_VEHICLE: vehicle.crash_vehicle,
        TerminationState.CRASH_OBJECT: vehicle.crash_object,
        TerminationState.CRASH_BUILDING: vehicle.crash_building,
        TerminationState.CRASH_HUMAN: vehicle.crash_human,
        TerminationState.CRASH_SIDEWALK: vehicle.crash_sidewalk,
        TerminationState.OUT_OF_ROAD: self._is_out_of_road(vehicle),
        TerminationState.SUCCESS: self._is_arrive_destination(vehicle),
        TerminationState.MAX_STEP: max_step,
        TerminationState.ENV_SEED: self.current_seed,
        # TerminationState.CURRENT_BLOCK: self.agent.navigation.current_road.block_ID(),
        # crash_vehicle=False, crash_object=False, crash_building=False, out_of_road=False,
arrive_dest=False,
    }

    # for compatibility
    # crash almost equals to crashing with vehicles
    done_info[TerminationState.CRASH] = (
        done_info[TerminationState.CRASH_VEHICLE] or
done_info[TerminationState.CRASH_OBJECT]
        or done_info[TerminationState.CRASH_BUILDING] or
done_info[TerminationState.CRASH_SIDEWALK]
        or done_info[TerminationState.CRASH_HUMAN]
    )

    # determine env return
    if done_info[TerminationState.SUCCESS]:
        done = True
        self.logger.debug(
            "Episode ended! Scenario Index: {} Reason:
arrive_dest.".format(self.current_seed),
            extra={"log_once": True}
        )
    if done_info[TerminationState.OUT_OF_ROAD] and self.config["out_of_road_done"]:
        done = True
        self.logger.debug(
            "Episode ended! Scenario Index: {} Reason:
out_of_road.".format(self.current_seed),
            extra={"log_once": True}
        )
    if done_info[TerminationState.CRASH_VEHICLE] and self.config["crash_vehicle_done"]:
        done = True
        self.logger.debug(
            "Episode ended! Scenario Index: {} Reason: crash vehicle
".format(self.current_seed),
            extra={"log_once": True}
        )
    if done_info[TerminationState.CRASH_OBJECT] and self.config["crash_object_done"]:
        done = True
        self.logger.debug(
            "Episode ended! Scenario Index: {} Reason: crash object
".format(self.current_seed),
            extra={"log_once": True}
        )
    if done_info[TerminationState.CRASH_BUILDING]:
        done = True
        self.logger.debug(
            "Episode ended! Scenario Index: {} Reason: crash building

```



```

        ".format(self.current_seed),
            extra={"log_once": True}
        )
    if done_info[TerminationState.CRASH_HUMAN] and self.config["crash_human_done"]:
        done = True
        self.logger.debug(
            "Episode ended! Scenario Index: {} Reason: crash
human".format(self.current_seed),
            extra={"log_once": True}
        )
    if done_info[TerminationState.MAX_STEP]:
        # single agent horizon has the same meaning as max_step_per_agent
        if self.config["truncate_as_terminate"]:
            done = True
            self.logger.debug(
                "Episode ended! Scenario Index: {} Reason: max step ".format(self.current_seed),
                extra={"log_once": True}
            )
    return done, done_info

```

Please note that in the Safe RL environment `SafeMetaDriveEnv`, the episode will not be terminated when vehicles crashing into objects or vehicles. This is because we wish to investigate the safety performance of a vehicle in an extremely dangerous environments. Terminating episodes too frequently will let the training becomes too hard to complete.

It is worth noting that there is a special termination condition *max_step*. It means if the agent exists in the scene for more than `env.config["horizon"]` steps, it will be terminated. The lifetime of an agent is called *agent episode*. In single agent environment, when the agent reaches *max_step*, the environment will be reset and thus the lifetime of the environment, “environment episode”, has the same length as *agent episode*. However, in Multi-agent environment, the controllable target vehicles consistently respawn in the scene if old target vehicles are terminated. The the environment will stop spawning new target vehicles if the length of *environment episode* exceeds `env.config["horizon"]`. As the environment has to wait for the termination of all existing agents, the length of *environmental episode* will be greater than `env.config["horizon"]` but less than $2 * \text{env.config["horizon"]}$, while the length of *agent episode* is always less than `env.config["horizon"]`.

Note: when an agent reaches *max_step* limit, the terminated signal received by `_,_,terminated,truncated,_,env.step(action)` will be False, but the truncated signal is True. To make the terminated signal=True when truncated is flagged, turn on `truncate_as_terminate` in `env_config`. However, we don’t encourage this behavior, as [the gymnasium asks RL environments explicitly distinguish the timeout \(truncation\) and done \(Termination\)](#).

The following example help elaborate on the timeout mechanism used in MetaDrive.

```
from metadrive.envs.metadrive_env import MetaDriveEnv

def test_horizon(horizon, truncate_as_terminate):
    env = MetaDriveEnv({
        "horizon": horizon,
        "log_level": 50,
        "truncate_as_terminate": truncate_as_terminate})
    o, _ = env.reset()
    try:
        for i in range(1, 1000):
            _, _, tm, tc, _ = env.step([0., 0.])
            if tm or tc:
                break
    finally:
        env.close()
    return tm, tc, i

tm, tc, epi_length = test_horizon(500, truncate_as_terminate=False)
# tc when step==500, and no tm signal
assert tc and not tm and epi_length==500

tm, tc, epi_length = test_horizon(500, truncate_as_terminate=True)
# tc and tm when step==500
assert tm == tc == True and epi_length==500

tm, tc, epi_length = test_horizon(None, truncate_as_terminate=False)
# no length limit in this case, and no tc and tm signals!
assert tm == tc == False and epi_length > 500
```

Step Information

The step information dict `info` contains rich information about current state of the environment and the target vehicle. The step info is collected from various sources such as the engine, reward function, termination function, traffic manager, agent manager and so on. We summarize the dict as follows:

```

{
    # Number of vehicles being overtaken by ego vehicle in this episode
    'overtake_vehicle_num': 0,

    # Current velocity in km/h
    'velocity': 0.0,

    # The current normalized steering signal in [-1, 1]
    'steering': -0.06901532411575317,

    # The current normalized acceleration signal in [-1, 1]
    'acceleration': -0.2931942343711853,

    # The normalized action after clipped who is applied to the ego vehicle
    'raw_action': (-0.06901532411575317, -0.2931942343711853),

    # Whether crash to vehicle / object / building
    'crash_vehicle': False,
    'crash_object': False,
    'crash_building': False,
    'crash': False, # Whether any kind of crash happens

    # Whether going out of the road / arrive destination
    # or exceeding the maximal episode length
    'out_of_road': False,
    'arrive_dest': False,
    'max_step': False,

    # The reward in this time step / the whole episode so far
    'step_reward': 0.0,
    'episode_reward': 0.0,

    # The cost in this time step
    'cost': 0,

    # The length of current episode
    'episode_length': 1
}

```

The content of this dict keeps updating, and thus the content above may be out of date. We encourage users to write customized data to this dict, so more status can be exposed to monitor the simulation even without visualization.

Customization

To compose your own reward, cost and termination function. Just make a new environment and override the `reward_function`, `cost_function`, and `termination_function` of the base environment class. You can also record more information in `step_info` returned by these functions and deliver it outside the simulator.

```
from metadrive.envs.metadrive_env import MetaDriveEnv

class MyEnv(MetaDriveEnv):

    def reward_function(*args, **kwargs):
        return -10, {"is_customized": True}

env=MyEnv()
env.reset()
_,r,_,_,info = env.step([0,0])
assert r==-10 and info["is_customized"]
print("reward: {}, `is_customized` in info: {}".format(r, info["is_customized"]))
env.close()
```

[INFO] Environment: MyEnv

[INFO] MetaDrive version: 0.4.3

[INFO] Sensors: [lidar: Lidar(), side_detector: SideDetector(), lane_line_detector: LaneLineDetector()]

[INFO] Render Mode: none

[INFO] Horizon (Max steps per agent): 1000

[INFO] Assets version: 0.4.3

[INFO] Known Pipes: glxGraphicsPipe

[INFO] Start Scenario Index: 0, Num Scenarios : 1

reward: -10, `is\_customized` in info: True